

MIB CHURCH

MANUAL TÉCNICO ENCICLOPÉDICO DO DESENVOLVEDOR & GUIA DE CÓDIGO

Documentação Exaustiva: Arquitetura, Banco de Dados, Componentes JSX, Funções JS e Regras RLS

CONTEÚDO TÉCNICO EXAUSTIVO DO MANUAL

- 1. Arquitetura da Aplicação e Stack Tecnológico (React 19, Supabase RLS, Vite, Tailwind)
- 2. Dicionário de Dados e SQL DDL das 18 Tabelas PostgreSQL no Supabase
- 3. Controlador Principal (App.jsx), Rotas, Sessão e Gerenciamento de Estado Global
- 4. Engenharia do Módulo de Pessoas (PessoasModulo.jsx, DetalhesMembro.jsx, Crop & Autocadastro)
- 5. Engenharia do Módulo de Células (CelulasModulo.jsx, Lançamento de Reuniões & Relatórios)
- 6. Engenharia do Módulo Financeiro (ModuloFinanceiro.jsx, Transações & Carnês QR Code)
- 7. Engenharia do Módulo de Escolas Teológicas e Algoritmo de Indicação de Líderes
- 8. Engenharia do Gestor Ministerial (MinisteriosManager.jsx, MinistryDetails.jsx & Funções)
- 9. Engenharia das Escalas Ministeriais (EscalasMinisteriais.jsx, Gerador Mensal & PNG Export)
- 10. Engenharia de Agenda, Eventos Públicos e Check-in por QR Code na Portaria
- 11. Engenharia do Módulo Espiritual (MuralOracao.jsx, QuizPersonalidade.jsx & Tributos)
- 12. Matriz de Segurança, SQL Supabase RLS e Regra Especial de Escala Ministerial
- 13. Guia de Operação, Deployment Otimizado (Vite Build) e Solução de Problemas (Troubleshooting)

ATRIBUIÇÃO FORMAL DE AUTORIA E RESPONSABILIDADE TÉCNICA

Jadson da Silva Viana

Desenvolvedor & Arquiteto Principal da Plataforma MIB Church

Edição Técnica Exaustiva · Agosto de 2026 · Versão Oficial 2.0

DECLARAÇÃO FORMAL DE AUTORIA E PROPRIEDADE INTELECTUAL

A presente apostila técnica consubstancia o manual definitivo de código fonte, arquitetura de software, estrutura de banco de dados e engenharia de requisitos da plataforma MIB Church.

REGISTRO FORMAL DE AUTORIA

Todos os módulos do código-fonte (componentes React JSX, scripts de utilitários JavaScript, estilização Tailwind CSS, consultas SQL Supabase, triggers e políticas de segurança RLS) possuem atribuição formal e soberana de autoria a Jadson da Silva Viana. Nenhuma parte deste sistema foi concebida sem sua direta intervenção arquitetural.

SUMÁRIO EXECUTIVO DA DOCUMENTAÇÃO TÉCNICA

Capítulo 1	Arquitetura da Aplicação e Stack Tecnológico
Capítulo 2	Dicionário de Dados e SQL DDL das 18 Tabelas PostgreSQL no Supabase
Capítulo 3	Controlador Principal (App.jsx), Rotas e Estado Global
Capítulo 4	Engenharia do Módulo de Pessoas e Autocadastro Público
Capítulo 5	Engenharia do Módulo de Células e Relatórios Semanais
Capítulo 6	Engenharia do Módulo Financeiro e Carnês com QR Code
Capítulo 7	Engenharia do Módulo de Escolas e Indicação de Líderes
Capítulo 8	Engenharia do Gestor Ministerial (Estruturas e Funções)
Capítulo 9	Engenharia das Escalas Ministeriais (Gerador & Exportação PNG)
Capítulo 10	Agenda de Eventos e Leitor de QR Code na Portaria
Capítulo 11	Módulo Espiritual, Pedidos de Oração e Temperamentos
Capítulo 12	Matriz de Segurança, SQL Supabase RLS e Regra Especial de Escalar
Capítulo 13	Guia de Operação, Deployment e Solução de Problemas (Troubleshooting)

CAPÍTULO 1: ARQUITETURA DA APLICAÇÃO E STACK TECNOLÓGICO

O MIB Church é uma Single Page Application (SPA) desenvolvida com a biblioteca React 19 em conjunto com o bundler Vite 8. O armazenamento de dados e autenticação são providos pelo Supabase PostgreSQL com segurança via Row Level Security (RLS).

1.1 Dependências e Bibliotecas (package.json)

- **React 19 & React-DOM 19:** Framework reativo responsável pelo renderizador virtual DOM e gerenciamento de estado.
- **@supabase/supabase-js v2:** Cliente oficial para requisições PostgreSQL REST e canais em tempo real.
- **@tailwindcss/vite v4:** Engine de estilização utilitária baseada em CSS de alta performance.
- **lucide-react:** Biblioteca de ícones vetoriais responsivos.
- **html2canvas / html-to-image:** Renderizador de elementos DOM para conversão de cartões em imagens PNG.
- **jspdf v4:** Geração dinâmica de relatórios e carnês em PDF no cliente.
- **html5-qrcode & qrcode.react:** Leitor de câmera em tempo real para leitura de ingressos e gerador de códigos QR.
- **recharts:** Biblioteca de gráficos vetoriais SVG para exibição de métricas financeiras e demográficas.

[CÓDIGO / ESTRUTURA]: Estrutura Completa do Código-Fonte (src/)

```

src/
% % % App.jsx                # Controlador Geral (Roteamento, Autenticação e Tema)
% % % main.jsx               # Ponto de Entrada da Aplicação
% % % supabaseClient.js      # Conexão com Supabase SDK
% % % PessoasModulo.jsx      # Diretório de Membros e Filtros
% % % DetalhesMembro.jsx     # Ficha Cadastral e Cortador de Foto
% % % FormularioCadastro.jsx # Formulário de Cadastro Interno
% % % PublicRegistrationForm.jsx # Formulário Público de Autocadastro
% % % CelulasModulo.jsx      # Gestão de Células
% % % ModallancarReuniao.jsx # Chamada Semanal e Presença de Células
% % % RelatorioSemanal.jsx   # Relatório Consolidado para Supervisores
% % % ModuloFinanceiro.jsx   # Dashboard Financeiro e Dízimos
% % % TransacoesFinanceiras.jsx # Tabela de Lançamentos de Caixa
% % % CarneGenerator.jsx     # Emissão e Baixa de Carnês com QR Code
% % % CategoriasFinanceiras.jsx # Plano de Contas
% % % EscolasModulo.jsx      # Cursos Teológicos e Indicação de Líderes
% % % AgendaModulo.jsx       # Calendário de Eventos da Igreja
% % % LeitorQRCodeEvento.jsx # Validador de Ingressos por Câmera
% % % ModuloUtilitarios.jsx  # Utilitários Diversos
% % % MuralOracao.jsx        # Mural de Pedidos de Oração
% % % QuizPersonalidade.jsx  # Teste de Temperamentos Humano
% % % ministerial/          # Módulo Ministerial
% % % EscalasMinisteriais.jsx # Escalas, Gerador Mensal & PNG Export
% % % MinisteriosManager.jsx # Gestão de Ministérios e Cores
% % % MinistryDetails.jsx    # Atribuição de Voluntários e Líderes

```

CAPÍTULO 2: DICIONÁRIO DE DADOS E SQL DDL DAS TABELAS SUPABASE POSTGRESQL

O banco de dados do MIB Church é composto por 18 tabelas relacionais com chaves primárias UUID, restrições UNIQUE e políticas RLS ativas.

2.1 Estrutura SQL DDL das Principais Tabelas**[CÓDIGO / ESTRUTURA]: Tabela 1: pessoas (Membros, Visitantes e Liderança)**

```

CREATE TABLE pessoas (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nome VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE,
  telefone VARCHAR(50),
  cpf VARCHAR(20) UNIQUE,
  rg VARCHAR(20),
  data_nascimento DATE,
  genero VARCHAR(20), -- 'masculino', 'feminino'
  estado_civil VARCHAR(30),
  endereco TEXT,
  bairro VARCHAR(100),
  cargo VARCHAR(50) DEFAULT 'membro', -- 'membro', 'lider-celula', 'lider', 'supervisor', 'diacono', 'presbitero',
  'pastor', 'admin',
  permissao VARCHAR(50) DEFAULT 'membro',
  foto_url TEXT,
  data_batismo DATE,
  batizado_espirito BOOLEAN DEFAULT false,
  celula_id UUID REFERENCES celulas(id),
  status VARCHAR(20) DEFAULT 'ativo',
  created_at TIMESTAMPTZ DEFAULT now()
);

```

[CÓDIGO / ESTRUTURA]: Tabela 2: células (Grupos Pequenos)

```
CREATE TABLE celulas (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  nome VARCHAR(255) NOT NULL,  
  dia_semana INTEGER, -- 0 (Dom) a 6 (Sáb)  
  horario VARCHAR(10),  
  endereco TEXT,  
  lider_id UUID REFERENCES pessoas(id),  
  colider_id UUID REFERENCES pessoas(id),  
  supervisor_id UUID REFERENCES pessoas(id),  
  status VARCHAR(20) DEFAULT 'ativa'  
);
```

[CÓDIGO / ESTRUTURA]: Tabela 3: reunioes_celula (Registros Semanais de Reunião)

```
CREATE TABLE reunioes_celula (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  celula_id UUID REFERENCES celulas(id) ON DELETE CASCADE,  
  data_reuniao DATE NOT NULL,  
  presentes_ids UUID[] DEFAULT '{}',  
  qtd_visitantes INTEGER DEFAULT 0,  
  valor_oferta NUMERIC(10,2) DEFAULT 0.00,  
  estudo_tema VARCHAR(255),  
  observacoes TEXT,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

[CÓDIGO / ESTRUTURA]: Tabela 4: financeiro_transacoes (Receitas e Despesas)

```
CREATE TABLE financeiro_transacoes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tipo VARCHAR(20) NOT NULL, -- 'receita', 'despesa'  
  categoria_id UUID REFERENCES financeiro_categorias(id),  
  descricao VARCHAR(255) NOT NULL,  
  valor NUMERIC(12,2) NOT NULL,  
  data_transacao DATE NOT NULL,  
  membro_id UUID REFERENCES pessoas(id),  
  forma_pagamento VARCHAR(50), -- 'pix', 'dinheiro', 'cartao', 'boleto'  
  status VARCHAR(20) DEFAULT 'confirmado'  
);
```

[CÓDIGO / ESTRUTURA]: Tabela 5: ministerio_membros (Equipes Ministeriais e Flag de Líder)

```
CREATE TABLE ministerio_membros (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ministerio_id UUID REFERENCES ministerios(id) ON DELETE CASCADE,  
  pessoa_id UUID REFERENCES pessoas(id) ON DELETE CASCADE,  
  funcao_id UUID REFERENCES funcoes_ministeriais(id),  
  lider BOOLEAN DEFAULT false, -- Flag essencial que libera permissão no Gestor Ministerial  
  UNIQUE(ministerio_id, pessoa_id)  
);
```

[CÓDIGO / ESTRUTURA]: Tabela 6: eventos_escala (Cultos e Programações de Escala)

```
CREATE TABLE eventos_escala (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  titulo VARCHAR(255) NOT NULL,  
  data_evento DATE NOT NULL,  
  hora_inicio VARCHAR(10),  
  hora_termino VARCHAR(10),  
  local VARCHAR(255),  
  fardamentos JSONB, -- Armazena a farda do dia por ministério  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

CAPÍTULO 3: CONTROLADOR PRINCIPAL (APP.JSX) E GERENCIAMENTO DE ESTADO GLOBAL

O arquivo `App.jsx` controla o ciclo de vida global da aplicação React, gerenciando a autenticação, carregamento do membro logado e controle de acesso via função `temPermissao`.

3.1 Mapeamento de Estados e Hooks em App.jsx

- **session & user:** Estado que armazena a sessão ativa de autenticação com Supabase Auth.
- **membroLogado:** Estado que guarda os dados cadastrais da tabela `pessoas` vinculada ao e-mail autenticado.
- **isLiderMinisterio:** Estado booleano que indica se a pessoa logada é líder em qualquer ministério na tabela `ministerio\_membros`.
- **moduloAtual:** Estado string determinando qual tela/módulo está visível (`overview`, `pessoas`, `celulas`, `financeiro`, `escolas`, `ministerial`, `agenda`, `utilitarios`, `configuracoes`).

[CÓDIGO / ESTRUTURA]: Algoritmo de Verificação de Permissão (App.jsx)

```
const temPermissao = (mod) => {  
  if (!membroLogado) return false;  
  const p = (membroLogado.permissao || membroLogado.cargo || '').toLowerCase();  
  
  if (p === 'admin' || p === 'pastor') return true;  
  if (mod === 'overview') return true;  
  if (mod === 'pessoas') return ['secretaria', 'lider', 'supervisor'].includes(p);  
  if (mod === 'celulas') return ['secretaria', 'lider-celula', 'lider', 'supervisor'].includes(p);  
  if (mod === 'financeiro') return ['tesouraria', 'tesoureiro'].includes(p);  
  if (mod === 'ministerial') return true; // Qualquer membro acessa visualmente; ações são filtradas internamente  
  return false;  
};
```

CAPÍTULO 4: ENGENHARIA DO MÓDULO DE PESSOAS E AUTOCADASTRO PÚBLICO

Compreende `PessoasModulo.jsx`, `DetalhesMembro.jsx`, `FormularioCadastro.jsx` e `PublicRegistrationForm.jsx`.

4.1 Recursos e Funções da Gestão de Membros

- **calcularPermissoesPorCargo(cargo):** Retorna o resumo textual formatado das atribuições de cada cargo.
- **uploadFotoStorage(file, membroid):** Executa o upload da imagem cortada em Canvas para o bucket `fotos-membros` no Supabase Storage.
- **tratarSubmitVisitante(e):** Processa o formulário de novos visitantes vinculando-os à célula mais próxima.

[CÓDIGO / ESTRUTURA]: Recorte de Imagem em Canvas (DetalhesMembro.jsx)

```
const getCroppedImg = async (imageSrc, pixelCrop) => {  
  const image = await createImage(imageSrc);  
  const canvas = document.createElement('canvas');  
  const ctx = canvas.getContext('2d');  
  
  canvas.width = pixelCrop.width;  
  canvas.height = pixelCrop.height;  
  
  ctx.drawImage(image, pixelCrop.x, pixelCrop.y, pixelCrop.width, pixelCrop.height, 0, 0, pixelCrop.width,  
    pixelCrop.height);  
  return new Promise((resolve) => {  
    canvas.toBlob((blob) => resolve(blob), 'image/jpeg', 0.95);  
  });  
};
```

CAPÍTULO 5: ENGENHARIA DO MÓDULO DE CÉLULAS E RELATÓRIOS SEMANAIS

Compreende `CelulasModulo.jsx`, `ModalLancarReuniao.jsx` e `RelatorioSemanal.jsx`.

5.1 Lançamento de Reuniões Semanais

- **lançarReuniaoSemanal()**: Insere o registro na tabela `reunioes\_celula` incluindo array de presença, visitantes e oferta.
- **calcularPresencaPorcento()**: Determina a taxa de assiduidade comparando os membros da célula com a lista de presentes marcados.

CAPÍTULO 6: ENGENHARIA DO MÓDULO FINANCEIRO E CARNÊS COM QR CODE

Compreende `ModuloFinanceiro.jsx`, `TransacoesFinanceiras.jsx` e `CarneGenerator.jsx`.

6.1 Geração de Carnês e Baixa por QR Code

[CÓDIGO / ESTRUTURA]: Impressão e Baixa de Parcela com QR Code (CarneGenerator.jsx)

```
const processarBaixaCarneQRCode = async (tokenQRCode) => {
  const { data, error } = await supabase
    .from('carnes_contribuicao')
    .update({ status: 'pago', data_pagamento: new Date() })
    .eq('qr_code_token', tokenQRCode)
    .select('*', pessoas(nome));

  if (data && data.length > 0) {
    alert(`Baixa efetuada com sucesso: ${data[0].pessoas.nome}`);
  }
};
```

CAPÍTULO 7: ENGENHARIA DO MÓDULO DE ESCOLAS TEOLÓGICAS E INDICAÇÃO DE LÍDERES

Compreende `EscolasModulo.jsx`. Administra a formação acadêmica e ministerial.

7.1 Algoritmo de Indicação Inteligente de Novos Líderes

[CÓDIGO / ESTRUTURA]: Seleção de Alunos de Destaque (EscolasModulo.jsx)

```
const indicarNovosLideres = (alunos, frequencia, notas) => {
  return alunos.filter(aluno => {
    const pctFrequencia = calcularFrequenciaAluno(aluno.id, frequencia);
    const mediaNota = calcularMediaNotasAluno(aluno.id, notas);
    return pctFrequencia === 100 && mediaNota >= 9.0 && !aluno.is_lider;
  });
};
```

CAPÍTULO 8: ENGENHARIA DO GESTOR MINISTERIAL (ESTRUTURAS E FUNÇÕES)

Compreende `MinisteriosManager.jsx` e `MinistryDetails.jsx`.

8.1 Gestão de Equipes e Atribuição de Líderes

- **vincularVoluntario()**: Conecta um membro a um ministério e função na tabela `ministerio\_membros`.
- **toggleLiderMinisterio()**: Atualiza a coluna `lider = true`, liberando a função de escalação para aquele usuário.

CAPÍTULO 9: ENGENHARIA DAS ESCALAS MINISTERIAIS (GERADOR & EXPORTAÇÃO PNG)

Compreende `EscalasMinisteriais.jsx`. Responsável pelo agendamento e gerador em lote.

9.1 Algoritmo de Geração Mensal em Lote

[CÓDIGO / ESTRUTURA]: Gerador de Eventos para o Mês Selecionado (EscalaMinisteriais.jsx)

```
const calcularPreviaEventos = () => {
  const eventosGerados = [];
  const diasNoMes = new Date(anoGeracao, mesGeracao + 1, 0).getDate();

  for (let dia = 1; dia <= diasNoMes; dia++) {
    const dataObj = new Date(anoGeracao, mesGeracao, dia);
    const diaSemana = dataObj.getDay(); // 0 (Dom) a 6 (Sáb)

    // Filtra cultos padrão configurados para este dia
    const cultosDoDia = cultosPadrao.filter(c => c.dayOfWeek === diaSemana);
    cultosDoDia.forEach(c => {
      eventosGerados.push({
        titulo: c.titulo,
        data_evento: dataObj.toISOString().split('T')[0],
        hora_inicio: c.time,
        hora_termino: c.timeEnd,
        local: localPadrao
      });
    });
  }
  setPreviaEventos(eventosGerados);
};
```

9.2 Exportação para Imagem PNG (html2canvas)**[CÓDIGO / ESTRUTURA]: Exportação Gráfica em PNG (EscalaMinisteriais.jsx)**

```
const handleExportarMensalPNG = async (download = true) => {
  const elem = document.querySelector('.mensal-export-page');
  if (!elem) return;

  const canvas = await html2canvas(elem, { scale: 2, useCORS: true });
  const imgData = canvas.toDataURL('image/png');

  if (download) {
    const link = document.createElement('a');
    link.download = `Escala_${filtroMes + 1}_${filtroAno}.png`;
    link.href = imgData;
    link.click();
  }
};
```

CAPÍTULO 10: AGENDA DE EVENTOS E LEITOR DE QR CODE NA PORTARIA

Compreende `AgendaModulo.jsx`, `PublicEventRegistration.jsx` e `LeitorQRCodeEvento.jsx`.

10.1 Leitor de Ingressos por Câmera**[CÓDIGO / ESTRUTURA]: Validação de Portaria por QR Code (LeitorQRCodeEvento.jsx)**

```
const validarQRCodePortaria = async (tokenQRCode) => {
  const { data, error } = await supabase
    .from('inscritos_eventos')
    .update({ status_presenca: 'confirmado', data_checkin: new Date() })
    .eq('qr_code_token', tokenQRCode)
    .select('*', { pessoas(nome) });

  if (data && data.length > 0) {
    setMensagemSucesso(`Check-in efetuado: ${data[0].pessoas.nome}`);
  } else {
    setMensagemErro('Ingresso inválido.');
```

CAPÍTULO 11: MÓDULO ESPIRITUAL, PEDIDOS DE ORAÇÃO E TEMPERAMENTOS

Compreende `MuralOracao.jsx`, `QuizPersonalidade.jsx` e `CalculadoraTributos.jsx`.

11.1 Teste de Temperamentos (QuizPersonalidade.jsx)

[CÓDIGO / ESTRUTURA]: Cálculo de Perfil Comportamental (QuizPersonalidade.jsx)

```
const calcularPontuacaoTemperamento = (respostas) => {  
  const pontos = { Sanguineo: 0, Colerio: 0, Melancolico: 0, Fleumatico: 0 };  
  respostas.forEach(r => { pontos[r.tipo] += r.peso; });  
  
  const dominante = Object.keys(pontos).reduce((a, b) => pontos[a] > pontos[b] ? a : b);  
  return { pontos, dominante };  
};
```

CAPÍTULO 12: MATRIZ DE SEGURANÇA, SQL SUPABASE RLS E REGRA ESPECIAL DE ESCALAR

A segurança dos dados é imposta pelas diretivas Row Level Security (RLS) no PostgreSQL.

+P REGRA ESPECIAL DE ESCALA MINISTERIAL (GESTOR MINISTERIAL)

Se um Membro ou Líder de Célula for cadastrado como Líder de algum Ministério na tabela `ministerio_membros` com `lider = true`, a variável `isLiderMinisterio` assume o valor `true` e a função "Escalar" no módulo de Escalas é automaticamente habilitada para ele.

[CÓDIGO / ESTRUTURA]: Lógica da Liberação da Função Escalar (EscalasMinisteriais.jsx)

```
const permissaoLower = (membroLogado?.permissao || membroLogado?.cargo || '').toLowerCase();  
const isPerfilRestrito = ['membro', 'lider-celula', 'lider', 'supervisor'].includes(permissaoLower);  
  
// Se o perfil for restrito MAS o usuário for líder de ministério, habilita a função "Escalar"  
const isMembroNormal = isPerfilRestrito && !isLiderMinisterio;
```

CAPÍTULO 13: GUIA DE OPERAÇÃO, DEPLOYMENT E SOLUÇÃO DE PROBLEMAS (TROUBLESHOOTING)

Instruções para compilação, execução e tratamento de exceções:

- **Modo de Desenvolvimento:** Executar `npm run dev -- --host` para iniciar o servidor local Vite.`
- **Build de Produção:** Executar `npm run build` para gerar o pacote estático otimizado em /dist`.`
- **Resolução de Erro 403:** Se o usuário encontrar permissão negada, execute as políticas RLS no console SQL do Supabase.

